МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей
(индекс по РУП и наименование профессионального модуля)

Специальность 09.02.07 Информационные системы и программирование

Обучающийся _____ Долдин Александр Дмитриевич
(подпись) (фамилия, имя, отчество)

Группа __ИПс-31____

Руководитель практической подготовки от профильной организации
_____
(должность)

_____ _____
(подпись) (фамилия, имя, отчество)

МП «13» ноября 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«13» ноября 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ....3
Глава 1. ОПИСАНИЕ ЗАДАНИЯ....6
1.1. Назначение модуля импорта товаров....6
1.2. Требования к CSV-файлу....6
1.3. Требования к функциональности....7
1.4. Ограничения к функциональности....8
Глава 2. АРХИТЕКТУРА РЕШЕНИЯ....10
2.1. Общая структура проекта....10
2.2. Роль модели Product....11
2.3. Роль репозитория в памяти....11
2.4. Роль сервиса импорта....11
2.5. Роль контроллера и представлений ASP.NET MVC....12
Глава 3. ФОРМАТ CSV-ФАЙЛА....13
3.1. Описание формата CSV....13
Глава 4. РЕАЛИЗАЦИЯ ФУНКЦИОНАЛЬНОСТИ....15
4.1. Как читается CSV-файл....15
4.2. Как выполняются проверки (Валидация)....17
4.3. Добавление товаров в память....17
4.4. Передача импорта в представление....18
Глава 5. ТЕСТИРОВАНИЕ И ОТЛАДКА....20
5.1. Разработанные тестовые сценарии....20
5.2. Unit-тесты....22
5.3. Отладка....23
ЗАКЛЮЧЕНИЕ....24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....26

ВВЕДЕНИЕ

Актуальность: В современной электронной коммерции скорость и удобство наполнения каталога товарами напрямую влияют на конкурентоспособность магазина. Ручной ввод данных о товарах (артикулы, цены, характеристики, остатки на складе) через веб-интерфейс является трудоемким процессом, который сопровождается высоким риском ошибок. Наиболее эффективным способом массовой загрузки данных является импорт из табличных форматов, в частности CSV-файлов.

Однако при импорте плохих данных (содержащих пропуски, некорректные цены, дубликаты артикулов) возникает риск нарушения целостности базы данных и падения приложения. В связи с этим актуальной является разработка программного модуля, который не только автоматизирует чтение CSV-файлов, но и обеспечивает строгую валидацию данных перед их загрузкой в систему, а также предоставляет пользователю наглядный отчет о результатах импорта.

Место прохождения практики. Производственная практика проходила в Пермском институте (филиале) федерального государственного бюджетного образовательного учреждения высшего образования «Российский экономический университет имени Г. В. Плеханова» в Приемной комиссии, в период с 03 ноября по 13 ноября 2026 года.

Цель практики: формирование практических умений по разработке требований к программному модулю, их интеграции в веб-приложение, отладке, тестированию и проверке качества кода в рамках профессионального модуля ПМ.02 «Осуществление интеграции программных модулей».

1. Для достижения поставленной цели в рамках индивидуального задания (Вариант 2 «Книги») были определены следующие задачи:
2. Проанализировать индивидуальное задание и определить требования к модулю импорта товаров.

3. Спроектировать модель данных (класс Product) с учетом предметной области «Книги».
4. Разработать структуру хранения данных в памяти на основе паттерна «Репозиторий».
5. Реализовать логику чтения и валидации данных из CSV-файла с сохранением корректных записей в коллекцию в оперативной памяти (без использования баз данных).
6. Реализовать логику валидации данных (проверка обязательных полей, форматов цен, года издания, дубликатов артикулов).
7. Интегрировать модуль импорта в ASP.NET Core MVC-приложение (создать контроллер и представления).
8. Выполнить отладку и разработать набор юнит-тестов с использованием фреймворка xUnit.
9. Провести инспектирование кода на соответствие базовым стандартам кодирования.
10. Оформить отчет по практике и подготовить проект к защите.

Объект исследования: процесс интеграции программных модулей в веб-приложения.

Предмет исследования: процессы интеграции программных модулей, методы обработки и валидации структурированных данных (формат CSV), а также технологии проектирования веб-приложений на платформе ASP.NET Core MVC с использованием языка программирования C#.

Инструменты разработки: язык программирования C# (.NET 8), фреймворк ASP.NET Core MVC, среда разработки Microsoft Visual Studio 2022, система контроля версий Git.

Структура отчета. Отчет состоит из введения, четырех разделов, заключения, списка использованных источников и приложений. В первом разделе описано задание и требования к модулю. Во втором разделе представлена архитектура решения. В третьем разделе описана реализация функциональности и работа с CSV. Четвертый раздел посвящен тестированию и отладке программного продукта.

Глава 1. ОПИСАНИЕ ЗАДАНИЯ

1.1. Назначение модуля импорта товаров

Разрабатываемый программный модуль предназначен для автоматизации процесса наполнения каталога электронного магазина товарами. В рамках выполнения индивидуального задания реализуется Вариант 2 «Книги».

Модуль позволяет пользователю (контент-менеджеру или администратору) загружать данные о товарах из текстового файла формата CSV, выполнять их программную проверку на корректность и сохранять валидные записи в оперативную память приложения для дальнейшего отображения в веб-интерфейсе.

1.2. Требования к CSV-файлу

Входные данные должны представлять собой текстовый файл в кодировке UTF-8 с расширением .csv. Первая строка файла является заголовочной и содержит имена колонок, разделителем полей выступает запятая. Файл должен содержать следующий набор колонок: sku, name, description, price, currency, category_name, stock, image_url, is_active, isbn, author, publisher, publication_year.

Требования к типам и обязательности данных (с учетом специфики варианта «Книги»):

1. sku – артикул (строка), обязательное поле.
2. name – название книги (строка), обязательно поле, не пустое.
3. description – описание (строка), может быть пустым.
4. price — цена (число с плавающей точкой), обязательное поле, строго больше 0.
5. currency — валюта (строка), если пусто — по умолчанию "RUB".

6. category_name — категория (строка), если пусто — по умолчанию "Без категории".
7. stock — остаток на складе (целое число), не меньше 0.
8. image_url — ссылка на картинку (строка), может быть пустым.
9. is_active — активность (логическое значение), допустимы значения true, false, 1, 0.
10. isbn — ISBN (строка), обязательное поле, должно содержать 10 или 13 символов (без учета дефисов).
11. author — автор (строка), обязательное поле.
12. publisher — издательство (строка), может быть пустым.
13. publication_year — год издания (целое число), обязательное поле, от 1900 до текущего года + 1.

1.3. Требования к функциональности

Разрабатываемый программный модуль должен обеспечивать выполнение следующих функций:

1. Чтение файла: загрузка и построчное чтение CSV-файла по указанному пользователем локальному пути.
2. Парсинг данных: преобразование строк CSV в объекты C# с поддержкой кавычек (если поле содержит запятую, оно должно быть корректно обработано).
3. Валидация: проверка каждой строки на соответствие требованиям (заполненность обязательных полей, формат чисел, диапазон года, уникальность артикула).
4. Обработка ошибок: формирование списка ошибок с указанием физического номера строки в файле и текстового описания проблемы.

Приложение не должно аварийно завершаться при передаче неверных данных.

5. Сохранение данных: добавление только валидных записей (у которых нет ошибок и чей артикул не является дубликатом) в коллекцию в памяти.
6. Вывод результатов: отображение на веб-странице статистики импорта (всего обработано строк, успешно импортировано, количество ошибок) и детального списка найденных ошибок.
7. Просмотр каталога: отображение списка успешно загруженных книг в виде HTML-таблицы.
8. Дополнительные функции (сверх базового задания): поиск товаров по названию, подсчет общего количества и суммарной стоимости товаров в каталоге.

1.4. Ограничения к функциональности

При разработке модуля установлены следующие ограничения:

1. Отсутствие базы данных: в соответствии с заданием, использование систем управления базами данных (СУБД) не предусмотрено. Все данные хранятся исключительно в оперативной памяти (в коллекциях List<T>).
2. Энергозависимость памяти: при перезапуске веб-приложения или пула приложений IIS все загруженные данные будут утеряны. Для учебного задания это является допустимым.
3. Локальный доступ к файлам: модуль считывает файлы по пути, доступному на сервере (в рамках задания — из папки wwwroot/csv/). Загрузка файлов напрямую с компьютера пользователя через веб-интерфейс не является обязательной (но может быть реализована как дополнительный функционал).

4. Стек технологий: реализация выполняется на языке C# с использованием фреймворка ASP.NET Core MVC (платформа .NET 8). Использование сторонних библиотек для парсинга CSV не допускается — парсер реализуется самостоятельно.

Глава 2. АРХИТЕКТУРА РЕШЕНИЯ

2.1. Общая структура проекта

Для реализации задачи был спроектирован проект ASP.NET Core MVC. Код организован по папкам в соответствии с паттерном MVC и принципом разделения ответственности (Separation of Concerns):

1. Models/ — содержит модели данных (Product, ImportResult, ImportError) и View-модели (ImportViewModel, ProductsIndexViewModel).
2. Services/ — включает в себя бизнес-логику: интерфейсы и реализацию репозитория (IProductRepository, InMemoryProductRepository), сервис импорта (IProductImportService, ProductImportService) и вспомогательный класс для разбора строк (CsvLineParser).
3. Controllers/ — содержит ProductsController, который обрабатывает HTTP-запросы.
4. Views/Products/ — представления (Razor-страницы) для ввода пути к файлу (Import.cshtml) и отображения каталога (Index.cshtml).
5. wwwroot/csv/ — папка для хранения тестовых CSV-файлов.
6. ECommerceCsvImport.Tests/ — отдельный проект для юнит-тестирования (xUnit).

Процесс обработки данных при импорте можно представить в виде следующей схемы:

CSV file

↓

ProductImportService

↓

Validation + Mapping to Product

↓

InMemoryProductRepository

↓

ProductsController

↓

ASP.NET MVC Views

2.2. Роль модели Product

Класс Product является центральной моделью предметной области. Он описывает сущность «Книга» и содержит базовые поля (артикул, название, цена, остаток и т.д.), а также специализированные поля для Варианта 2: Isbn, Author, Publisher и PublicationYear. Модель используется сервисом импорта для создания объектов и репозиторием для их хранения, а также передается в представления для отображения пользователю.

2.3. Роль репозитория в памяти

Для хранения данных используется паттерн «Репозиторий». Интерфейс IProductRepository задает контракт (добавление, получение списка, поиск по артикулу), а класс InMemoryProductRepository реализует его, сохраняя данные в коллекцию List<Product>.
В файле Program.cs репозиторий зарегистрирован как Singleton (AddSingleton). Это гарантирует, что в течение всего времени работы приложения (между разными HTTP-запросами) используется один и тот же экземпляр репозитория, и данные в памяти не теряются при переходе со страницы импорта на страницу каталога.

2.4. Роль сервиса импорта

Сервис импорта (ProductImportService) инкапсулирует в себе всю сложную бизнес-логику обработки файлов. Он отвечает за проверку существования файла, его чтение, вызов парсера для разбивки строк, валидацию полученных значений, проверку дубликатов артикулов и

сохранение валидных объектов в репозиторий. Сервис возвращает объект ImportResult, содержащий статистику и список ошибок.

2.5. Роль контроллера и представлений ASP.NET MVC

Контроллер ProductsController выступает связующим звеном. Он не содержит бизнес-логики и логики хранения. Его задача — принять запрос от браузера, вызвать соответствующий метод сервиса или репозитория и передать полученные данные в представление. Представления (Views) отвечают исключительно за визуализацию. Import.cshtml отображает форму ввода пути и результаты импорта (таблицу с ошибками), а Index.cshtml — список успешно загруженных книг.

Глава 3. ФОРМАТ CSV-ФАЙЛА

3.1. Описание формата CSV

Входные данные для импорта представляют собой текстовый файл формата CSV с разделителем-запятой. Первая строка файла является заголовочной. Пример файла sample_products.csv для варианта «Книги»:

sku,name,description,price,currency,category_name,stock,image_url,is_active,isbn,author,publisher,publication_year 2001,C# для начинающих,Учебник по программированию,1500,RUB,Программирование,10,/images/csharp.jpg,true,978-5-17-123456-7,Иванов И.И.,Эксмо,2022

Описание полей и правила их валидации представлены в таблице 1.

Таблица 1 — Формат и правила валидации полей CSV-файла

Поле	Тип данных	Обязательность	Описание и правила валидации
sku	string	Обязательно	Артикул товара. Не может быть пустым. Проверяется на уникальность
name	string	Обязательно	Название книги. Не может быть пустым.
description	string		Описание. Может быть пустым.
price	decimal	Обязательно	Цена товара. Должна быть числом больше 0.
currency	string	Необязательно	Валюта. RUB по умолчанию.
category_name	string	Необязательно	Категория. Без категории по умолчанию.
stock	int	Обязательно	Остаток на складе. Целое число не меньше 0.
image_url	string	Необязательно	Ссылка на изображение.
is_active	bool	Обязательно	Статус. Допустимые значения: true, false, 1,

			0.
isbn	string	Обязательно	ISBN книги. Должен содержать 10/13 символов.
author	string	Обязательно	Автор книги. Не может быть пустым.
publisher	string	Необязательно	Издательство. Может быть пустым.
publication_year	int	Обязательно	Год издания. Целое число от 1900 до текущего года +1.

4.1. Как читается CSV-файл

Процесс чтения начинается в контроллере ProductsController при POST-запросе. Пользователь передает виртуальный путь (например, /csv/sample_products.csv). С помощью метода ResolveFilePath контроллер преобразует его в физический путь на диске (добавляя путь к папке wwwroot).

Далее вызывается метод сервиса ImportFromCsv(string filePath). Внутри него выполняется проверка существования файла, после чего файл считывается целиком в массив строк с помощью метода File.ReadAllLines(filePath).

Считанные строки передаются в метод ImportFromLines, где происходит их построчный разбор. Для преобразования строки в набор отдельных значений используется разработанный класс CsvLineParser. Он разбивает строку по запятым, корректно обрабатывая пустые поля и игнорируя запятые внутри двойных кавычек.

CsvLineParser.cs | RepositoryTests.cs | ProductImportServiceTests.cs | Cs

ECommerceCsvImport

```
namespace ECommerceCsvImport.Services
{
    Ссылок: 5
    public static class CsvLineParser
    {
        Ссылок: 5
        public static IReadOnlyList<string> Parse(string line)
        {
            var result = new List<string>();
            if (line == null)
                return result;

            var current = new System.Text.StringBuilder();
            bool inQuotes = false;

            for (int i = 0; i < line.Length; i++)
            {
                char c = line[i];

                if (inQuotes)
                {
                    if (c == '"')
                    {
                        if (i + 1 < line.Length && line[i + 1] == '"')
                        {
                            current.Append('"');
                            i++;
                        }
                        else
                        {
                            inQuotes = false;
                        }
                    }
                    else
                    {
                        current.Append(c);
                    }
                }
                else
                {
                    if (c == '"')
                    {
                        inQuotes = true;
                    }
                    else if (c == ',')
                    {
                        result.Add(current.ToString().Trim());
                        current.Clear();
                    }
                    else
                    {
                        current.Append(c);
                    }
                }
            }

            result.Add(current.ToString().Trim());

            return result;
        }
    }
}
```

Рисунок 1 — метод Parse

4.2. Как выполняются проверки (Валидация)

Проверка данных выполняется в методе ImportFromLines сервиса ProductImportService. Логика реализована через последовательную проверку каждого поля.

Если поле не проходит проверку, в локальный список ошибок rowErrors добавляется объект ImportError, содержащий физический номер строки в файле и текст ошибки. Проверяются следующие условия: пустое значение SKU или названия, некорректный формат цены (или цена ≤ 0), отрицательный остаток на складе, неверное логическое значение is_active, длина ISBN, отсутствие автора, а также выход года издания за допустимый диапазон (от 1900 до текущего года + 1).

Если после проверки всех полей список rowErrors не пуст, строка считается невалидной. Все найденные ошибки переносятся в общий результат ImportResult, а товар не сохраняется в память.

```
int maxYear = DateTime.Now.Year + 1;
string yearStr = GetValue("publication_year");
int publicationYear = 0;
bool yearValid = int.TryParse(yearStr, out publicationYear)
                 && publicationYear >= 1900
                 && publicationYear <= maxYear;
if (!yearValid)
{
    rowErrors.Add(new ImportError
    {
        RowNumber = rowNumber,
        Message = $"Год издания должен быть целым числом от 1900 до {maxYear}"
    });
}
```

Рисунок 2 — Проверка года издания

4.3. Добавление товаров в память

Если при валидации строки не возникло ни одной ошибки, сервис проверяет артикул на дубликат (сравнивая с уже существующими товарами в репозитории и с теми, что были добавлены в текущем файле). Если дубликата

нет — создается новый объект Product, заполняются все его свойства, и он передается в метод Add() репозитория InMemoryProductRepository.

Поскольку репозиторий зарегистрирован как Singleton, созданный объект Product сохраняется в коллекции List<Product> в оперативной памяти и становится доступен для других частей приложения до момента перезапуска сервера.

4.4. Передача импорта в представление

Метод ImportFromLines возвращает объект ImportResult, содержащий общее количество обработанных строк, количество успешно добавленных книг и список всех ошибок.

Контроллер ProductsController принимает этот объект и передает его в модель представления ImportViewModel. Далее контроллер возвращает представление Import.cshtml. В Razor-разметке с помощью условного оператора @if (Model.Result != null) проверяется наличие результата, после чего на странице отрисовываются сводная таблица статистики и таблица со списком ошибок (если они есть).

При переходе на страницу списка книг (/Products/Index), контроллер вызывает _repository.GetAll(), передает список книг в ProductsIndexViewModel, и представление Index.cshtml с помощью цикла @foreach выводит все загруженные данные в итоговую HTML-таблицу.

ECommerceCsvImport Home Privacy

Книги

Всего товаров: **11**

Общая стоимость: **21 000 RUB**

Поиск по названию... Найти Сбросить

Артикул	Название	Цена	Категория	Остаток	Активен	ISBN	Автор	Издательство	Год
2001	C# для начинающих	1 500 RUB	Программирование	10	Да	978-5-17-123456-7	Иванов И.И.	Эксмо	2022
2002	ASP.NET MVC в действии	2 200 RUB	Программирование	5	Да	978-5-17-765432-1	Петров П.П.	Питер	2023
2003	Чистый код	1 800 RUB	Программирование	8	Да	978-5-17-111111-2	Сидоров С.С.	Вильямс	2021
2004	Алгоритмы и структуры данных	2 500 RUB	Информатика	3	Нет	978-5-17-222222-3	Кузнецов К.К.	БХВ-Петербург	2020
2005	Паттерны проектирования	1 900 RUB	Программирование	7	Да	978-5-17-333333-4	Фёдоров Ф.Ф.	Питер	2019
2006	Базы данных	1 700 RUB	Базы данных	6	Да	978-5-17-444444-5	Михайлов М.М.	Эксмо	2022
2007	Веб-разработка на JavaScript	2 100 RUB	Веб-разработка	4	Да	978-5-17-555555-6	Алексеев А.А.	Вильямс	2023
2008	Python для анализа данных	1 600 RUB	Программирование	9	Да	978-5-17-666666-7	Николаев Н.Н.	БХВ-Петербург	2022
2009	Сети и телекоммуникации	1 400 RUB	Сети	2	Нет	978-5-17-777777-8	Орлов О.О.	Питер	2021
2010	Операционные системы	2 300 RUB	Информатика	5	Да	978-5-17-888888-9	Смирнов С.С.	Эксмо	2020
2011	Архитектура компьютера	2 000 RUB	Информатика	11	Да	978-5-17-999999-0	Волков В.В.	Вильямс	2019

Импортировать книги

Рисунок 3 — Список книг

Импорт книг из CSV

Путь к CSV-файлу

/csv/sample_products.csv

Например: /csv/sample_products.csv

Импортировать книги

Результат импорта

Показатель	Значение
Всего строк	11
Успешно импортировано	11
Ошибки	0

Перейти к списку книг

Рисунок 4 — Страница импорта

Глава 5. ТЕСТИРОВАНИЕ И ОТЛАДКА

5.1. Разработанные тестовые сценарии

Согласно требованиям к программному модулю, было разработано шесть тестовых сценариев. Тестирование проводилось вручную с использованием браузера (Google Chrome) и заранее подготовленных CSV-файлов. Основная цель - проверить, что приложение корректно обрабатывает как валидные данные, так и ошибочные, не завершаясь аварийно.

В таблице 2 представлены разработанные тестовые сценарии.

Таблица 2 - Тестовые сценарии для модуля импорта

№	Сценарий	Входные данные	Ожидаемый результат
1	Успешный импорт	sample_products.csv (1111 корректных книг)	1111 товаров добавлены, ошибок нет
2	Неверная цена (отрицательная)	invalid_price.csv (цена -500)	Ошибка по строке 2: «Поле price должно быть числом больше 0»
3	Неверный ISBN	invalid_isbn.csv (ISBN = 12345)	Ошибка: «Поле isbn обязательно и должно содержать 10 или 13 символов»
4	Отсутствует автор	missing_author.csv	Ошибка: «Поле author обязательно»
5	Неверный год издания	invalid_year.csv (год 1800)	Ошибка: «Год издания должен быть целым числом от 1900 до ...»
6	Дубликат артикула	duplicate_sku.csv (два товара с SKU 2001)	Первый добавлен, второй пропущен. Ошибка: «Артикул товара уже существует»
7	Пустой файл	empty_file.csv (0 байт)	Приложение не падает, выводится сообщение «Данные для импорта отсутствуют»
8	Неверне значение is_active	bad_active.csv (значение yes)	Ошибка: «Поле is_active должно иметь значение true, false, 1 или 0»



Рисунок 5 — Неверная (отрицательная) цена

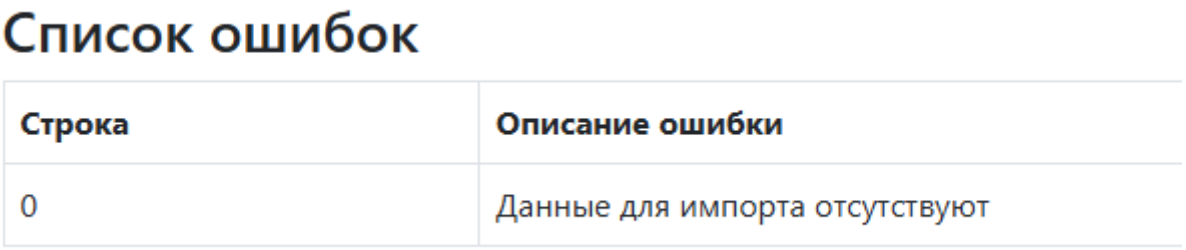


Рисунок 6 — Пустой файл

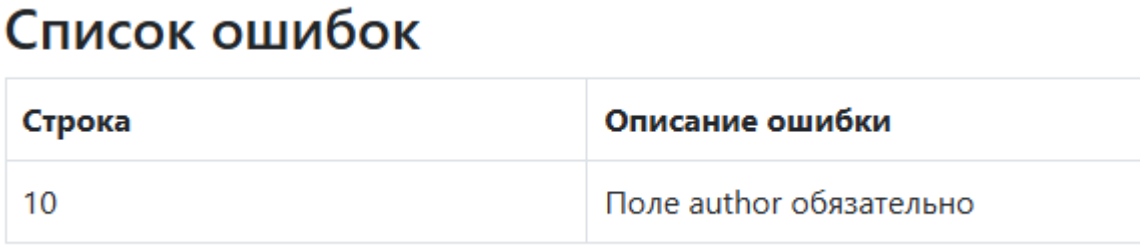


Рисунок 7 — Пустое поле «author»

5.2. Unit-тесты

Помимо ручного тестирования через веб-интерфейс, в рамках задания был разработан проект модульных тестов (юнит-тестов) с использованием фреймворка xUnit (проект EcommerceCsvImport.Tests).

Тесты покрывают три ключевых уровня приложения:

1. Тесты репозитория (RepositoryTests): проверяют корректность добавления товаров, поиск по артикулу (в том числе без учета регистра и пробелов) и работу с пустым репозиторием.
2. Тесты парсера (CsvLineParserTests): проверяют разбор строк с пустыми полями и корректную обработку запятых внутри кавычек.
3. Тесты сервиса импорта (ProductImportServiceTests): проверяют логику валидации (цены, ISBN, года, дубликатов) и формирование объекта ImportResult.

Все разработанные тесты успешно проходят (статус Passed / зеленый цвет).

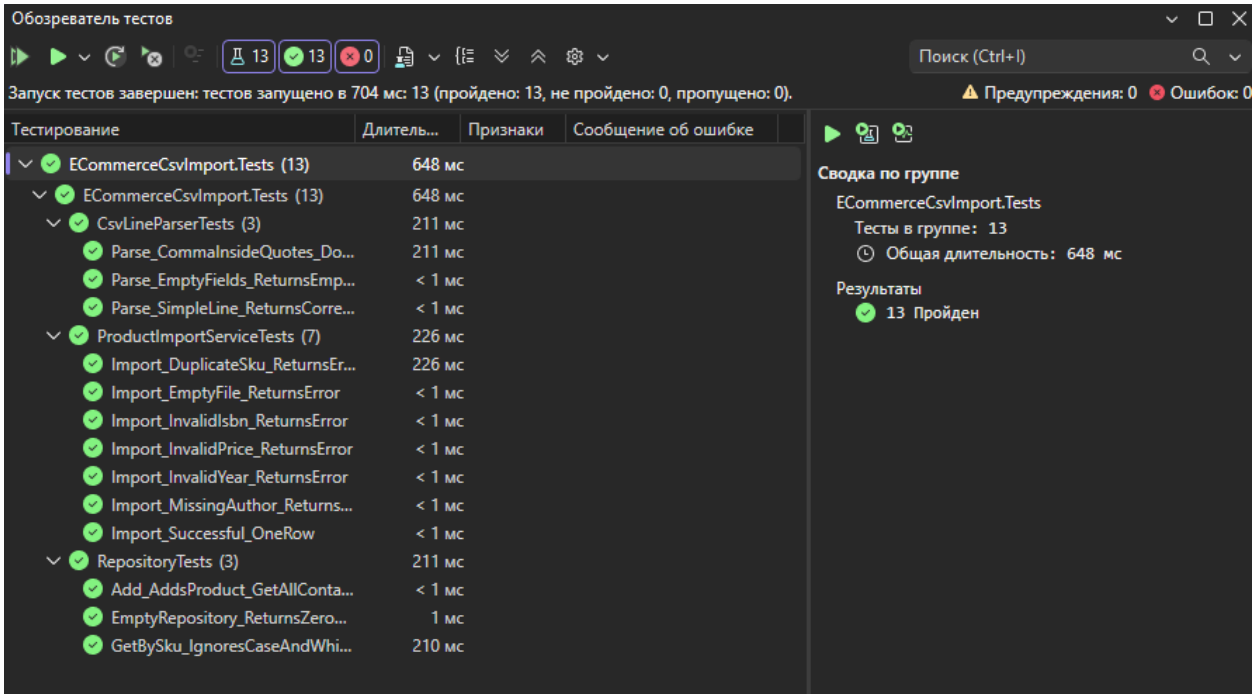


Рисунок 8 — Обозреватель тестов

5.3. Отладка

В процессе разработки использовались стандартные средства отладки Visual Studio (IDE): точки останова (breakpoints) для пошагового выполнения кода (Step Into / Step Over) при разборе строк CSV и просмотр значений переменных через окна Locals и Watch. Это позволило выявить и устранить ошибки, связанные с парсингом пустых строк и обработкой исключений при чтении файлов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения индивидуального задания производственной практики был разработан программный модуль импорта товаров из CSV-файлов в веб-приложение ASP.NET Core MVC (Вариант 2 «Книги»). По итогам работы можно сделать следующие выводы.

1. В процессе работы были закреплены и усовершенствованы практические навыки разработки веб-приложений на языке C#. Была спроектирована архитектура приложения с разделением ответственности (паттерн MVC), изучен механизм внедрения зависимостей (Dependency Injection) в ASP.NET Core, а также настроен жизненный цикл сервисов (использование Singleton для хранения данных в памяти). Кроме того, был получен опыт самостоятельной разработки алгоритма парсинга текстовых данных (CSV) с поддержкой экранирования кавычек, реализации строгой валидации бизнес-данных, написания юнит-тестов (xUnit) и работы с системой контроля версий Git.

2. В ходе разработки возникли определенные сложности. Основная трудность заключалась в корректной обработке CSV-файлов: стандартное разбиение строки по запятой приводило к ошибкам, если в названии книги или описании встречалась запятая. Также на начальном этапе возникали сложности с настройкой жизненного цикла репозитория — при использовании стандартного внедрения (Scoped) данные обнулялись при переходе между страницами.

3. Проблема с парсингом была решена путем написания собственного алгоритма посимвольного чтения строки (класс CsvLineParser), который отслеживает открывающие и закрывающие кавычки. Проблема с потерей данных была решена регистрацией репозитория как Singleton в файле Program.cs, а также добавлением блокировок (lock) для обеспечения потокобезопасности при одновременных запросах.

4. Разработанный модуль является учебным и может быть масштабирован. В дальнейшем в проект можно добавить:

1. Базу данных (например, SQL Server или PostgreSQL): для постоянного хранения товаров, чтобы данные не терялись после перезапуска сервера (с заменой InMemoryProductRepository на репозиторий, работающий через Entity Framework Core).
2. Асинхронный импорт: использование методов ReadAllLinesAsync и асинхронных контроллеров для того, чтобы интерфейс не «зависал» при загрузке очень больших файлов (десятки тысяч строк).
3. Загрузку файла через браузер: реализация формы с IFormFile, чтобы пользователю не приходилось вручную вводить путь к файлу, а можно было просто выбрать его с компьютера.
4. Пагинацию (постраничный вывод): для удобного просмотра каталога, если количество товаров станет слишком большим.
5. Расширенный поиск и фильтрацию: добавление фильтрации по категориям, авторам и сортировки по цене или году издания.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Албахари, Дж. C# 10 и .NET 6. Современная кроссплатформенная разработка / Дж. Албахари, Б. Албахари. - 6-е изд. - Москва : ДМК Пресс, 2022. - 1008 с.

2. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. - Санкт-Петербург : Питер, 2016. - 368 с.

3. Макконнелл, С. Совершенный код. Практическое руководство по разработке программного обеспечения / С. Макконнелл. - 2-е изд. - Москва : Русская редакция, 2010. - 896 с.

4. Троелсен, Э. Язык программирования C# 7 и платформы .NET и .NET Core / Э. Троелсен, Ф. Джепикс. - 8-е изд. - Москва : Вильямс, 2018. - 1360 с.

5. Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер. - Москва : Вильямс, 2006. - 544 с.

6. Фримен, А. ASP.NET Core MVC с примерами на C# для профессионалов / А. Фримен. - 8-е изд. - Москва : Диалектика, 2022. - 1008 с.

7. Документация по ASP.NET Core [Электронный ресурс] // Microsoft Learn. - Режим доступа: https://learn.microsoft.com/ru-ru/aspnet/core/ (дата обращения: 10.11.2026).

8. Документация по языку C# [Электронный ресурс] // Microsoft Learn. - Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/csharp/ (дата обращения: 10.11.2026).

9. Паттерн Repository (Репозиторий) [Электронный ресурс] // Refactoring.Guru. - Режим доступа:

https://refactoring.guru/ru/design-patterns/repository (дата обращения: 10.11.2026).